

Token Bucket Algorithm (Rate Limiting)

The **Token Bucket Algorithm** is one of the most widely used rate limiting techniques in distributed systems.

Used by major companies such as:

- Amazon
- Stripe

It is simple, memory efficient, and supports short traffic bursts.

1. Core Idea

A bucket stores tokens up to a fixed capacity.

None

Requests need tokens to proceed.

1 request = consumes 1 token

No token = request rejected / delayed

Tokens are replenished at a constant rate over time.

2. How It Works

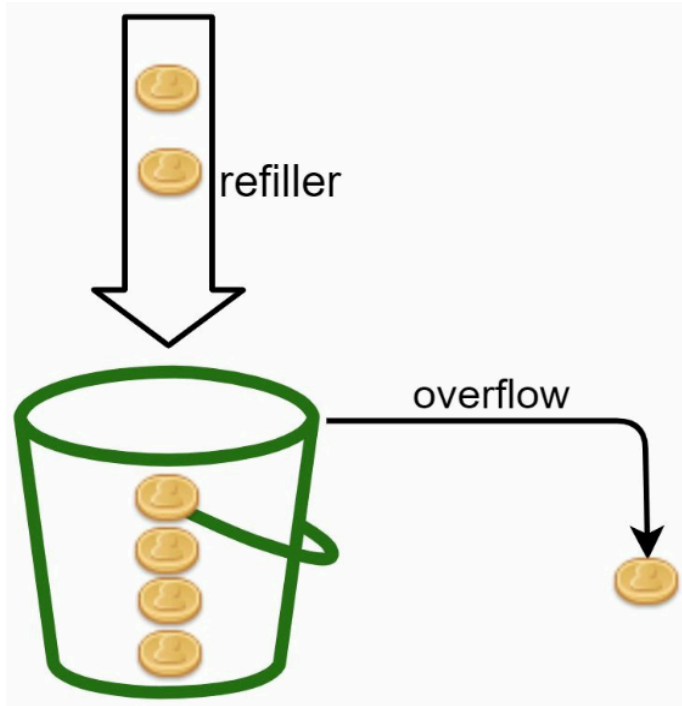
Step 1: Bucket Has Fixed Capacity

The token bucket has a predefined maximum size.

Example:

None

Bucket capacity = 4 tokens



As mentioned in **Figure**:

- Refiller adds tokens periodically
- If bucket is already full, extra tokens overflow and are discarded

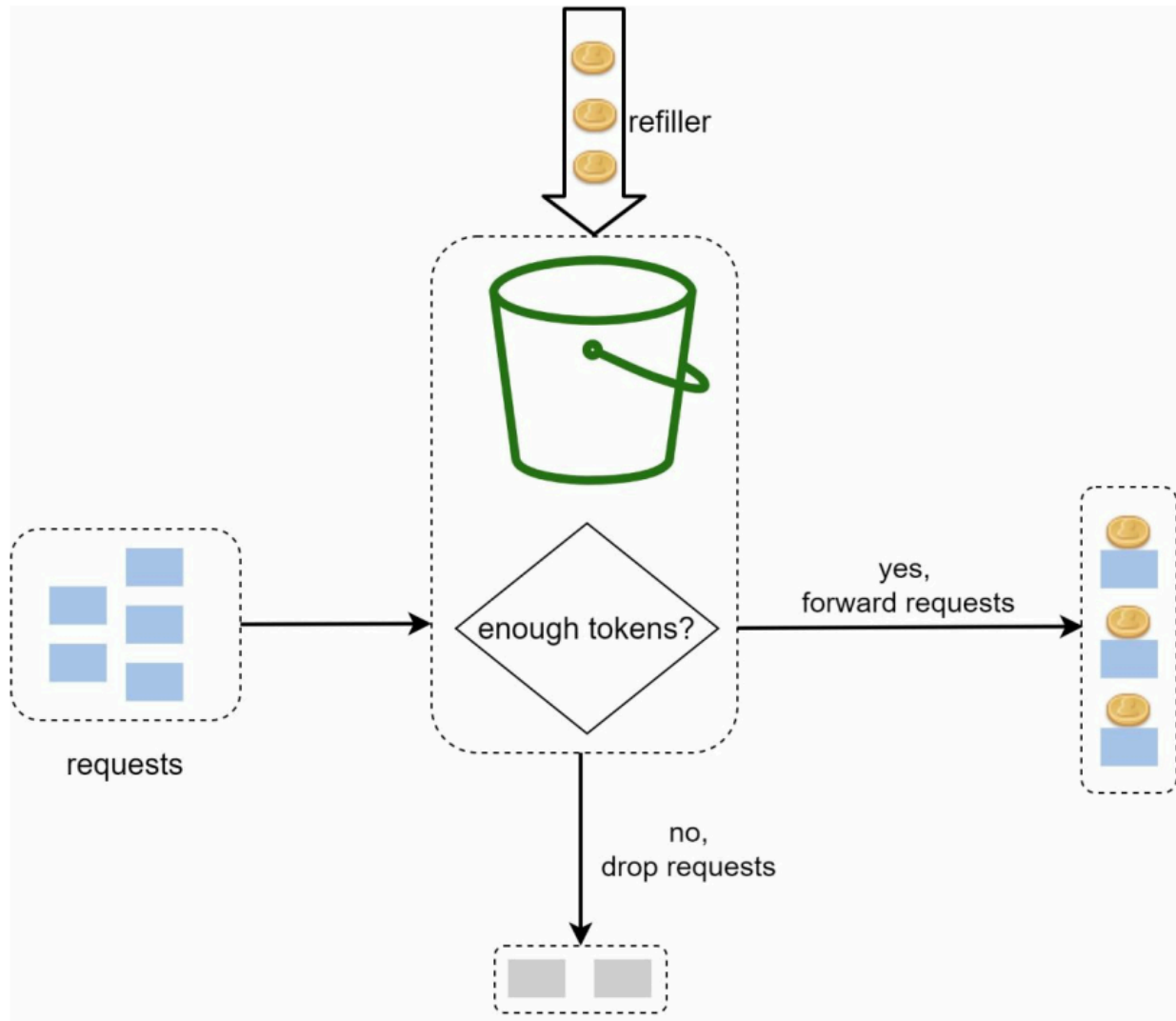
Step 2: Request Arrives

When a request comes:

None

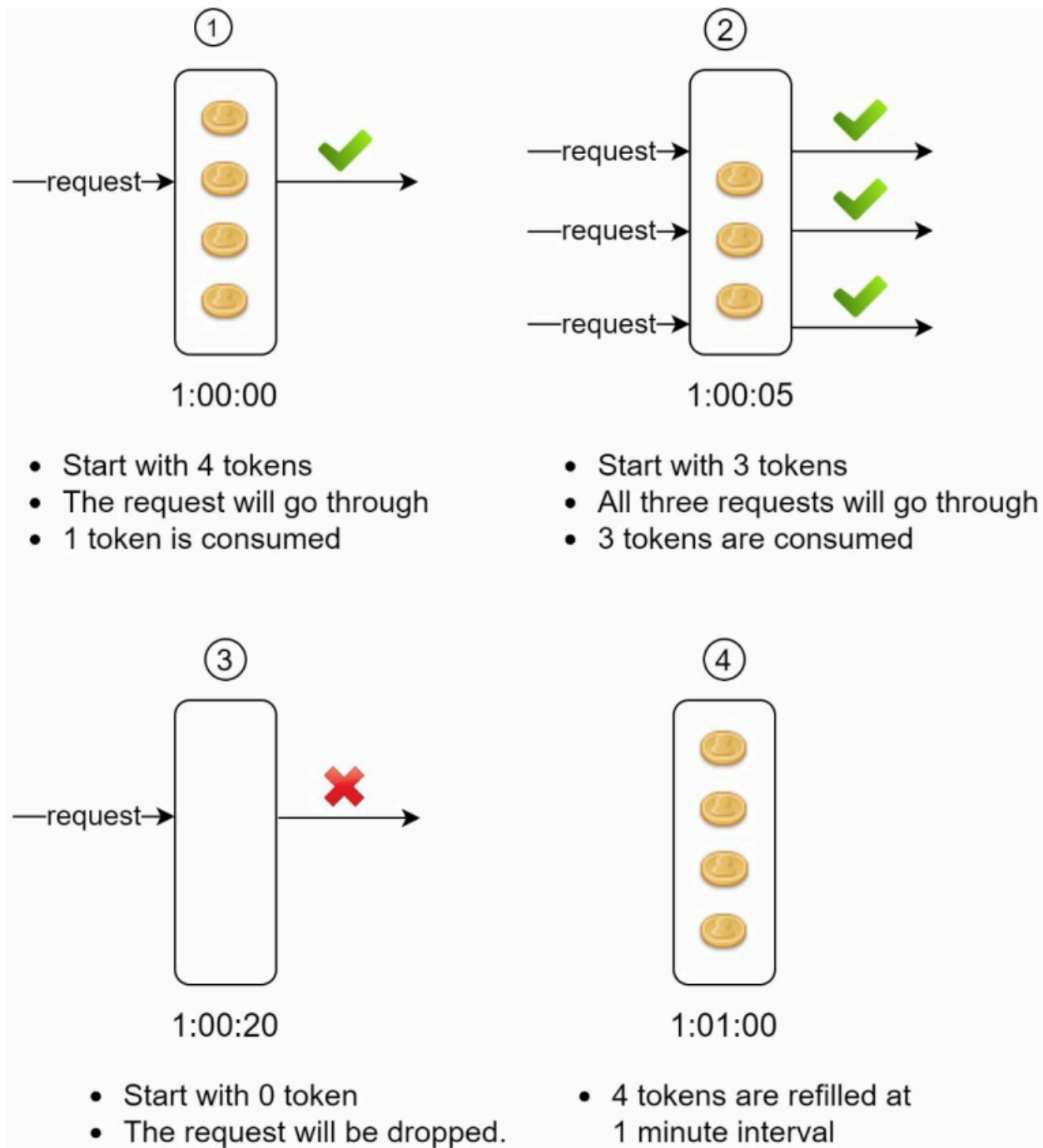
If tokens available → allow request

If no tokens → reject / throttle request



As described in **Figure**.

Step 3: Continuous Refill + Consumption



As shown in **Figure**:

Example:

None

Bucket size = 4

Refill rate = 4 tokens / minute

Requests consume tokens while refill gradually restores capacity.

3. Two Important Parameters

Bucket Size

Maximum number of tokens allowed in bucket.

None

Controls burst capacity

Larger bucket = more temporary burst traffic allowed.

Refill Rate

Number of tokens added per second/minute.

None

Controls sustained request rate

Higher refill = more throughput allowed.

4. How Many Buckets Are Needed?

Depends on rate limiting rules.

✓ Per API Endpoint

Example:

None

POST tweet = 1 req/sec

Add friends = 150/day

Like posts = 5 req/sec

Need separate buckets for each rule.

✓ Per User

Each user gets their own token bucket.

✓ Per IP Address

Each IP gets separate bucket.

Useful for abuse prevention.

✓ Global Bucket

Example:

None

System max = 10,000 req/sec

One shared global bucket for all traffic.

5. Example Flow

None

Time 0s → 4 tokens

2 requests arrive → 2 left

2 more requests → 0 left

Next request → blocked

Refill happens → tokens restored

6. Why It Is Popular

✓ Allows Burst Traffic

If bucket is full:

None

User can send multiple requests instantly

Useful for real-world traffic spikes.

✓ Simple Implementation

Works well with:

- API Gateways
- Reverse Proxies
- Distributed Services

✓ Memory Efficient

Only need:

- Token count
- Last refill timestamp

7. Advantages

- Easy to implement
- Memory efficient
- Supports bursts
- Good user experience
- Widely adopted in production systems

8. Disadvantages

- Choosing correct bucket size can be difficult
- Choosing proper refill rate requires tuning
- Distributed synchronization can be complex across multiple servers

None

Wrong parameters = Too strict or too lenient rate limits

9. System Design Use Cases

Used in:

- API rate limiting
- Login throttling
- Payment APIs
- SMS/email send limits
- Per-user request quotas

10. Interview Deep Dive Points

If asked implementation:

None

Use Redis INCR + EXPIRE

or Atomic Lua script in Redis

For distributed systems:

- Centralized Redis counter
- Sharded buckets
- Local + global hybrid limiting

Mental Model

None

Bucket size = Burst allowance

Refill rate = Long-term throughput

One-Line Summary

Token Bucket Algorithm limits request rate by allowing requests only when tokens are available, while tokens refill at a fixed rate—making it ideal for scalable systems that need fairness plus burst tolerance.